

Orizon Agents — Week 3 Tranche Submission

Proof of Deliverables

Programme	Stellar Instawards (Cohort 2026)
Milestone	M3 · Week 3 — Dispute Window & Partial-Credit Refund (Deliverable D3, Epic 4)
Sprint week	Mon 2026-09-21 → Fri 2026-09-25
Network	Stellar testnet only
Team	Danielle Bagaforo Meer — lead engineer (GitHub ALGOREX-PH) Rieselle Saure ("Rie") — PM + QA (GitHub rie-hash14)
Evidence captured	2026-09-26 — from the live testnet API, public GitHub and Stellar Expert pages, and a local run of the shipped interface against the test suite's fixtures. Every page states which of these it is, and its source URL.

DEPLOYED TESTNET CONTRACT IDS

AgentRegistry	CAPHXWU53UZUJGV7IAE57NNMH3YYB5MTW06YA53KKMXSFVL0ITBJ3GQ
ReputationLedger	CDCS0BEVZUPQZV56V4D6KYHZCLNGW2KXY74RUHSZ3EZUXF34DPW422ZT
PaymentEscrow	CBJPTMAPM60DGZCZ2IMEQSRUX3WGUXNMKDTNN2KMJ3NF6YZ50J5525PI
AttestationRegistry	CBYUZK0ET43UXTBXZUJIBBJW50DG2J2AZVVXCR3QONG0CAH0XQQHEGK
Asset SAC	CDLZFC3SYJDZT7K67VZ75HPJVIEUVNIXF47ZG2FB2RMQVU2HHG6CYS

EVIDENCE IN THIS DOCUMENT

–	What we shipped in Week 3	summary	p. 2
–	Where D3 stands, stated plainly	summary	p. 3
–	How to check any of this without trusting us	summary	p. 4
01	The dispute window and the dispute endpoint	4.02 · D3	p. 5
02	The settler-executed partial-credit refund	4.03 · D3	p. 6
03	The negative on-chain rating an upheld dispute writes	4.04 · D3	p. 7
04	What a dispute receipt needs in order to be truthful	4.06 · D3	p. 8
05	The dispute action on the buyer's trace view	4.05 · D3	p. 9
06	The dispute status and refund receipt the buyer reads	4.06 · D3	p. 10
07	Backend hardening: the money path, disclosure and settlement ratings	4.07 · hardening	p. 11
08	Frontend hardening: the receipt stops stating what it cannot show	4.07 · hardening	p. 12
09	Backend pull requests merged this week	PRs	p. 13
10	Frontend pull requests merged this week	PRs	p. 14
11	Independent QA: 268 authored commits in the public UAT repository	QA	p. 15
12	The live service is on testnet, against the deployed contracts	live API	p. 16
13	The platform's signer is the ledger's authorised scorer	live API	p. 17
14	All six dispute routes are served by the deployed API	live API	p. 18
15	The refund path is switched off on the deployment	live state	p. 19
16	A real refund transfer on testnet — from QA's drill, not the deployment	testnet tx	p. 20
17	The matching negative rating on testnet — same drill, same caveat	testnet tx	p. 21
18	Why no dispute can be raised on the deployment today	blocker	p. 22
19	The root cause, at the contract level	blocker	p. 23
20	The defect register: 27 defects logged this week	QA	p. 24
21	The dispute action, with the window open	UI · fixture	p. 25
22	Raising a dispute on one step	UI · fixture	p. 26
23	An open dispute, waiting on the platform	UI · fixture	p. 27
24	A credited dispute: refund and rating, both confirmed	UI · fixture	p. 28
25	The state this epic exists to get right	UI · fixture	p. 29
26	A rejected dispute, with the platform's reason	UI · fixture	p. 30

Every page names the public URL it was captured from, so each claim can be checked live. Pages 21–26 are the shipped interface rendered locally against test fixtures, because the dispute path cannot be reached on the deployment yet; the transaction hashes in them exist on no ledger. The two Stellar Expert pages are real testnet transactions from QA's drill, not from the deployed service. Both limits are stated again on the pages themselves.

What we shipped in Week 3

This week we built **Deliverable D3 — the dispute window and partial-credit refund**: a buyer who pays for an agent workflow gets 24 hours to dispute any single step, and an upheld dispute credits part of that step's charge back to their wallet and writes a permanent negative rating against the agent on-chain.

The five stories, all merged to `main`

- **4.02 — Dispute window and endpoint.** Every settlement stamps a 24-hour window on its own record. The payer opens a dispute against one step by signing a challenge with the wallet that paid; one dispute per step is a database rule, not a UI rule.
- **4.03 — Settler-executed partial-credit refund.** The platform pays a configured share of the disputed step's charge from its own wallet, clamped three ways, with the mutex taken *before* the signature so no crash or second adjudicator can pay twice.
- **4.04 — Negative on-chain rating.** An upheld dispute records the consequence on the ReputationLedger under a per-step derived job id, and that rating feeds the routing floor.
- **4.05 — Dispute action on the trace view.** The buyer sees each step's charge, what is creditable, a countdown, and a Dispute button — offered to the paying wallet alone.
- **4.06 — Dispute status and refund receipt.** Status, refund transfer and rating, each with its explorer link, and nothing shown as done until the chain confirms it.

Then we attacked it — story 4.07

Before declaring the epic done we audited everything it produced, on the rule that nothing counted as a finding until it had been reproduced against the running service. It found a **reproducible double payment**, a stale decision that could leave a dispute permanently unpayable, a misconfiguration that parked a payment mid-flight, the buyer's own words readable by a caller who had proved nothing, a flood that could deny every buyer their dispute window, a boot failure that printed secrets into the deploy log, a rating collision that lost half the ratings when one agent served two steps, and receipt copy that claimed a transfer existed when none had been submitted. Every fix carries a test that was written first and watched failing, and was then re-checked by putting the bug back.

The week in numbers

- **14 pull requests** merged across the public repositories; 1,295 commits; net +45,936 / -1,152 on `main`.
- **908 authored commits** by the lead engineer (687 backend, 221 frontend), 849 of them touching exactly one file; **268 authored commits** of independent QA work, every one touching exactly one file.
- Backend **2,177 tests** at 92.53% coverage; frontend **1,503 unit** and **149 end-to-end** tests; CI green on `main` in both repositories.

Where D3 stands, stated plainly

D3 is **built, merged and independently QA'd. Its evidence run on the live deployment has not been captured**, for two separate reasons — and the first one is not configuration.

1 · No workflow settles on the deployment

A dispute window is stamped on a **settlement**. On the deployed service, `PaymentEscrow.charge` attempts the transfer from an account that never signs, so the buyer's funds do not move and no run records a settlement — which means no window is ever stamped and there is nothing to dispute. QA filed it as D-050 (backend issue #67) behind the contract-level cause D-039 (contracts issue #3). This is the gating item: until it is fixed, no deployment or environment work produces a disputable run.

2 · The refund path is switched off, on an older build

Asking the deployment to uphold a dispute answers `503 dispute_refunds_disabled`, and its published API description shows a build from before the hardening merge. Both are shown in this document.

What we do have, and what it is not

QA drove the **real backend code** through one uphold and it produced two real, confirmed testnet transactions: a refund transfer and a `kind="dispute"` rating, one ledger apart. They prove the path executes and yields verifiable artifacts. They are **not** D3: the asset is a test asset rather than USDC, the ledger is QA's own drill rather than the platform's, and the source account is a developer machine's rather than the deployment's signer — which is checkable on the Stellar Expert pages included here. Every screenshot of the interface in this document is likewise a local run against the test suite's fixtures, labelled as such on its own page, with hashes that exist on no ledger.

The remaining path to D3, in order

1. Fix the escrow charge so a paid run settles (contracts issue #3).
2. Deploy the current `main`, which carries the hardening pass.
3. Set an operator key and switch the refund path on; confirm the database is configured so settlements survive an idle spin-down; leave the refund ceiling at a finite value; decide the refund asset.
4. Capture one run on the deployment producing both transactions, with the screen recording taken in the same session.

The written bundle this PDF accompanies sets all of this out with links: `Week-3-Tranche-Submission/03-deliverable-D3-dispute-refund.md`.

How to check any of this without trusting us

Every claim in this document resolves to something public: a merged pull request, a commit, a GitHub issue, a transaction on Stellar Expert, or a live endpoint you can call yourself.

Live endpoints

Check	Call
Network and contract ids	GET orizon-agents-be-stellar.onrender.com/api/stellar/network
Signer is the ledger's scorer	GET orizon-agents-be-stellar.onrender.com/readiness
The six dispute routes exist	GET orizon-agents-be-stellar.onrender.com/openapi.json
The refund switch is off	POST orizon-agents-be-stellar.onrender.com/api/disputes/{id}/uphold

The service runs on a free tier and sleeps; the first request can take 30–60 seconds.

Testnet transactions in this document

Artifact	Transaction hash
Refund transfer (QA drill)	a5baac432b582787df0a632b3bc12916c51e12575a8f77bf267fd45b728701b8
Dispute rating (QA drill)	7138e4e36e47f4f4404b2212aad5584d2f8fb941b387da76acae4c3b4cc07184

Each opens at stellar.expert/explorer/testnet/tx/<hash>. Both are drill transactions, as their pages in this document explain.

Independent QA

QA works in its own public repository, [BL0cksmiths/Orizon-Agents-UAT-Stellar](https://github.com/BL0cksmiths/Orizon-Agents-UAT-Stellar), with its own Playwright suites, drills and evidence pages. This week it logged **27 defects** (D-050 to D-076) — 25 as public GitHub issues, two money-path defects held privately — of which **eleven were found against our own hardening build**, including one regression our disclosure fix introduced. Its verdict on the dispute story card is **no-go**, and this submission reports it as no-go.

The code

Backend github.com/BL0cksmiths/Orizon-Agents-BE-Stellar · frontend github.com/BL0cksmiths/Orizon-Agents-FE-Stellar · QA github.com/BL0cksmiths/Orizon-Agents-UAT-Stellar · contracts github.com/BL0cksmiths/Orizon-Agents-Smart-Contract-Stellar · reference agent github.com/BL0cksmiths/Orizon-Agents-Example-Agent-Stellar. Design records for this epic are in the backend repository under [docs/decisions/](#) (ADRs 0002, 0007, 0008, 0009) and the operator runbook is [docs/disputes.md](#).

The dispute window and the dispute endpoint

Every settlement now stamps a **24-hour dispute window** on its own record at the moment it settles. The window is never recomputed from configuration, so changing the policy cannot move a deadline a buyer has already been told.

A buyer opens a dispute against **one step** of the workflow they paid for by signing a one-time challenge with the wallet that paid — no account, no password, no support ticket. One dispute per (job, step) is enforced by a database constraint, so a double-submit is answered with the first dispute unchanged rather than an error the buyer cannot act on.

Source	https://github.com/B10cksmiths/Orizon-Agents-BE-Stellar/pull/60
Captured	2026-09-26 19:48 PHT · screenshots/01-be-pr-60-dispute-window.png

Platform · Solutions · Resources · Open Source · Enterprise · Pricing

Search

Sign in · Sign up

B10cksmiths · Orizon-Agents-BE-Stellar

Notifications · Fork · Star

<> Code · Issues · Pull requests · Actions · Projects · Security and quality · Insights

4.02 — Dispute window state and the dispute endpoint #60

ALGOREX-PH merged 113 commits into main from feat/4.02-dispute-window 5 days ago

Conversation · Commits 113 · Checks 1 · Files changed 19 · +5,870 -25

ALGOREX-PH commented 5 days ago

Collaborator

Closes [4.02 \(BLO-30\)](#) — the dispute window and the dispute endpoint, built against 4.01's decision (D-002, ADR 0002).

A buyer can now raise a dispute against a settled step inside a bounded window, proving they are the payer with their wallet, and the record survives a restart.

The blocker this story had to solve first

Nothing a dispute is judged against was persisted anywhere. The `job_id` was a local inside `__settle_onchain`, the payer was a parameter of `__run`, and there has never been a per-step charged amount or a settlement timestamp. `state.tasks` could not hold it either: it is in-memory, it is lost on restart, and it **evicts terminal tasks first** — which is exactly the set a buyer disputes.

So a durable settlement record is part of this story, not a follow-up. `__run` writes one after the charge and the seal land and **before** `__submit_ratings`, because the ratings are a sequential run of on-chain submits and a process that dies partway through them would otherwise take the buyer's only evidence of what they paid for with it.

What ships

Area	File
Durable records	<code>app/services/dispute_store.py</code> — Postgres with an in-memory fallback, behind the <code>seam binding_store</code> established
Settlement capture	<code>app/services/execution_svc.py</code> — <code>__record_settlement</code> , best-effort, cannot fail the run
Dispute rules	<code>app/services/dispute_svc.py</code> — every refusal, in check order
Payer proof	<code>app/services/external_binding.py</code> — a third challenge purpose beside bind and unbind
API	<code>app/routers/disputes.py</code> — 4 routes
Docs	<code>docs/decisions/0007-dispute-window.md</code> , <code>docs/disputes.md</code> , dated amendment to ADR 0002

Routes: `POST /api/disputes/challenge`, `POST /api/disputes`, `GET /api/disputes/{dispute_id}`, `GET /api/tasks/{task_id}/disputes`.

Decisions worth reviewing

The payer proves themselves with a wallet signature, not the capability token the card proposed.
The token is a fine capability for reading a trace, but it cannot bear an authorization that moves money. `state.task.tokens` is a process-local dict, so it does not survive the restart the dispute window is measured against, and it proves possession of the *task link*, not that you are the wallet that paid. Reasoning and the rejected alternative are in ADR 0007 D2.

The duplicate rule is enforced by a partial unique index, not a read-then-insert. A read-then-insert inside one transaction does not prevent a double dispute at READ COMMITTED — both transactions snapshot before either commits and both insert. The index is `UNIQUE (job_id_hex, step_index) WHERE opening: partial`, because `dispute.events` is append-only and a total unique index would reject the dispute's own status transitions. Covered by a concurrency test that was verified to fail against a Python-side guard.

The reason is validated before the signature is checked, which inverts the order the card lists. Validating it is pure-fact and discloses nothing, and it is the only refusal a buyer can fix and retry — while the proof step consumes the challenge, so a buyer refused for an empty reason after the proof was spent would need a fresh nonce and a second wallet signature to resend the same dispute. Pinned by `test_an_empty_reason_is_refused_before_the_proof_is_spent`.

R12 is in force on this path. The dispute record's construction site names the replay guard and states that 4.04 must write the rating under `refund_svc.dispute_job_id{job_id}` — so the dispute is not born un-writable on-chain, which is this card's last acceptance criterion.

Honest limits

- A settlement that cannot be written leaves the run **paid but undisputable**. That is logged at ERROR and said plainly in the buyer's own trace rather than passed over in silence.
- `GET /api/tasks/{task_id}/disputes` is gated by the task read token, which lives in memory — so after a restart that listing answers as though the task were unknown, even though the records survived. Nothing a buyer needs is lost: opening a dispute is signature-gated and `GET /api/disputes/{dispute_id}` keeps working. Documented in `docs/disputes.md`.
- Without `DATASTORE`, the store is in-memory and the window is a promise that ends at the next restart. It says so at startup and logs any record it drops.

Verification

- `ruff check` `ruff format` `check` `mypy` — clean
- `pytest --cov` — **1692 passed, 2 skipped, 91.72%** (floor 82). Baseline 1534, so **+158 tests**.
- `check_contract_drift.py` — PASS, all 5 ids across both networks

Nothing on-chain changed and no contract was redeployed, so R9 is untouched.

Reviewers

No reviews

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestones

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

1 participant

ALGOREX-PH added 30 commits 5 days ago

added the dispute window setting

87534a3

added the credited fraction setting

f0770f3

documented the dispute settings in the env example

dcae12b

added the settlement and dispute records

ce015db

added the dispute store seam

8be0b2e

added the in-memory dispute store

35d8ef9

added the dispute store accessors

edcaba9

added the dispute window decision record

9ab0291

tracked which plan steps delivered

6d11a61

recorded the 24 hour window and the stamped deadline

e05b3a6

Orizon Agents — Week 3 Tranche Submission · Proof of Deliverables · Stellar testnet

5 / 30

The settler-executed partial-credit refund

An upheld dispute pays a configured share of the disputed step's charge back to the buyer's wallet, signed by the platform's settler from the platform's own funds. The amount is clamped to the smallest of three figures: the creditable amount frozen when the dispute opened, the step's settled price times the policy share, and the total actually settled.

The safety property that matters: a **claim row is written before the transfer is signed**, not after. A crash, a retry, a restart or a second adjudicator therefore cannot produce a second transfer, and a claim left over a payout whose outcome is unknown blocks further attempts instead of releasing them.

Source

<https://github.com/BI0cksmiths/Orizon-Agents-BE-Stellar/pull/62>

Captured

2026-09-26 19:48 PHT · screenshots/02-be-pr-62-partial-credit-refund.png

The screenshot shows a GitHub pull request interface. At the top, the repository is 'BI0cksmiths/Orizon-Agents-BE-Stellar'. The pull request title is '4.03 — Settler-executed partial-credit refund (land on main) #62'. It is marked as 'Merged' and shows 139 commits. The description explains the purpose of the PR and lists the changes brought to main. A detailed list of 83 commit messages follows, detailing the implementation of the refund process, including database schema changes, API key requirements, and testing. The interface also shows a sidebar with 'Reviewers', 'Assignees', 'Labels', 'Projects', 'Milestones', 'Development', and 'Participants'.

4.03 — Settler-executed partial-credit refund (land on main) #62

ALGOREX-PH merged 139 commits into main from feat/4.03-partial-credit-refund 5 days ago

Conversation 0 Commits 139 Checks 1 Files changed 19 +6,002 -91

ALGOREX-PH commented 5 days ago

Lands 4.03 (BLO-31) on main. The full description, design decisions and verification are on #61.

Why a second PR

#61 was opened stacked on feat/4.02-dispute-window, on the assumption that #60 was still open and GitHub would retarget it to main when #60 merged. #60 had in fact already merged, and GitHub only retargets a stacked PR when its base branch is deleted — so merging #61 put 4.03 into a branch that had already been merged, and none of it reached main.

This PR carries exactly the reviewed work, unchanged: feat/4.03-partial-credit-refund has the same tree as the post-#61 feat/4.02-dispute-window, and it merges into main without conflicts. There is nothing new to review here.

What it brings to main

- POST /api/disputes/{id}/uphold and /reject, behind a fail-closed require_adjudicator
- the settler-funded refund, idempotent via a durable refund_claims mutex, capped by MAX_REFUND_USDC, never retried automatically on an unconfirmed transfer
- ADR 0008, docs/disputes.md, and scripts/uphold_dispute.py for the live testnet evidence
- 19 files, 1880 tests passing, 91.97% coverage on the branch

Deploy notes

- The refund path ships off. Enabling it needs both DISPUTE_REFUNDS_ENABLED=true and API_KEY in the Render dashboard — the process refuses to boot with the switch on and no key.
- The schema change is idempotent (ALTER TABLE ... ADD COLUMN IF NOT EXISTS note) and runs on first use, so no manual migration is needed against the existing dispute_events table.
- Render does not auto-deploy from this org, so a manual deploy is needed after merge.

ALGOREX-PH added 30 commits 5 days ago

- added the dedicated refund cap setting
- required an api key whenever the refund path can sign
- added the refund path master switch
- narrowed the refund exposure to an enabled refund path
- documented the refund exposure in the validator docstring
- documented the refund settings in the env sample
- added the crediting state to the dispute lifecycle
- added the refund claim to the store protocol
- added the refund claim release to the store protocol
- claimed a refund in the in-memory store
- released a refund claim in the in-memory store
- added the refund claims table
- created the refund claims table with the others
- added the refund claim statements
- claimed a refund in the postgres store
- released a refund claim in the postgres store
- dropped the refund claim when a dispute finishes
- taught the fake pool about the refund claims table
- route the refund claim statements in the fake pool
- modelled the refund claims table in the fake pool
- held the refund claims in the fake pool state
- counted the third table in the lazy schema test
- scoped the append-only assertion to the event table
- reformatted the append-only assertion
- added the refund refusal error type
- logged every refused credit through one helper
- added the fail-closed adjudicator guard
- computed the creditable amount from the settlement
- refused a credit above the refund cap
- added the typed refund outcome

83 hidden items
Load more...

ALGOREX-PH added 27 commits 5 days ago

- stated which way the refund mutex fails and how it is cleared
- asserted the live verdict is read back off the dispute record
- asserted no line of output can carry a secret
- required an api key whenever refunds are switched on

The negative on-chain rating an upheld dispute writes

The refund makes the buyer whole; the rating is what changes who gets hired next. An upheld dispute writes a permanent kind="dispute" rating against the agent on the ReputationLedger, and that rating feeds the routing floor the planner applies before a buyer is asked to pay.

The ledger's replay guard keys on (agent_id, job_id), so a dispute rating is written under a **per-step derived job id**: the first half of the sealed job id, then a hash carrying the step index. A reviewer can still match every rating to its job by eye on Stellar Expert, because the first sixteen hex characters are the sealed job's own.

Source	https://github.com/BI0cksmiths/Orizon-Agents-BE-Stellar/pull/63
Captured	2026-09-26 19:48 PHT · screenshots/03-be-pr-63-dispute-rating.png

Platform · Solutions · Resources · Open Source · Enterprise · Pricing

Search

Sign in · Sign up

BI0cksmiths / Orizon-Agents-BE-Stellar

Notifications · Fork · Star

<> Code · Issues 17 · Pull requests 8 · Actions · Projects · Security and quality · Insights

4.04 — Negative on-chain rating for an upheld dispute #63

ALGOREX-PH merged 346 commits into main from feat/4.04-dispute-rating · 5 days ago

Conversation · Commits 346 · Checks 1 · Files changed 33 · +10,637 · -179

ALGOREX-PH commented 5 days ago

Closes [4.04 \(BLO-32\)](#): an upheld dispute now costs the agent reputation, not just one refund. This closes the loop between D2 (reputation-gated routing) and D3 (disputes).

Targets [a1a1](#) and contains [4.03](#). 4.04 was branched off 4.03 while [a#62](#) was still open. Merge order doesn't matter: once [a#62](#) merges, this diff shrinks to 4.04 alone (21 files). If this merges first, it brings 4.03 with it. It deliberately does not target a feature branch, because that is how [a#61](#) ended up stranded.

What changed

After an upheld dispute is **credited**, the settler writes a 'dispute-kind' rating of 10/100 to the ReputationLedger. The agent's 'dispute_rate_bps' rises, its score falls, and the next plan routes on the new score.

The obstacle, and why it turned out to be the answer

ReputationLedger.submit checks the replay guard Rated(agent_id, job_id) before it reads kind. The settler has already auto-rated every step under that exact pair, so the dispute rating is written under a **derived** job id.

Once that id is unique per dispute, the guard does the idempotency work too. Refunds needed a durable mutex because the chain can't stop a second transfer. Ratings don't, because the contract itself refuses a second rating under a key it has already seen. So retrying a rating is always safe, and retrying a refund never is.

The derived id

dispute_job_id(job, step) = job[:8] || sha256(job || "orizon-dispute:v1" || step)[:8]

- **The first half is the sealed job's own bytes.** SOW §6.1 needs a reviewer on Stellar Expert to tie the rating to the attested job without reading our code, and here they can see the link directly
- **It replaces 4.01's** sha256(job || "dispute")[:16]. That version hashed only the job, so when one agent served two steps of a job, the second upheld dispute was refused as a replay of the first. No rating had ever been written under the old scheme, so replacing it cost nothing.
- **It can't collide with the job's own key.** A match would need the hash to reproduce the job's tail by chance (2⁻⁶⁴), and the derivation checks for it and refuses loudly.
- **The derivation is permanent from now on**, because the ledger remembers every key it has seen. Golden vectors, recomputed independently of the function, pin it.

Decisions

Weight	The settler's own rating_weight_stroops(step.price.usdc). The card said both "settled value" and "consistent with every other rating", but every other rating uses the quoted price by design, so consistency won.
Order	The rating runs only after credited. A failed rating never touches the refund. Upholding a credited dispute again retries the rating only .
Replay	A replay when an earlier attempt is on record means that attempt landed. A replay with nothing on record is treated as a collision : a loud ERROR naming the agent, the dispute and the derived id.
How a caller learns the rating didn't land	Through the record, never an exception: credited with rating.tx null. An exception would turn a paid refund into a 5xx. The API needed no change.
Cache	invalidate(key) with a per-key generation, so a read already in flight can't write the pre-dispute score back, and a later caller can't join that stale read.

Found and fixed during the build

- **A cache invalidation could undo itself.** A read that started before the rating landed, and finished after the invalidation, wrote the old score straight back. A per-key generation now fences it.
- **A stale failure was fenced too.** Otherwise the first reader after an invalidation got the pre-dispute error back for the 2.5 s that failed reads are cached for.
- **A misconfigured deployment would have reported "unconfirmed" forever.** With reputation off or no ledger set, every retry came back as a timeout. It now checks rating_writer.config.get(), first and names the missing setting.
- **The operator script monkeypatched a service function** to see the ledger's answer. It now reads it through an explicit, optional on_rating observer on uphold. The HTTP route passes nothing, so an API response still always matches a later GET of the dispute.

Known limits

- **Cache invalidation reaches the current process only.** That's enough at workers 1, but with a second worker the other processes would serve the old score for up to the 15 s read TTL.
- **The settler's earlier auto-rating is never withdrawn**, so a dispute is a second, separate record next to it rather than a correction. Together the two average to 15-52.5.
- **A rating that landed after a submit that returned no hash** shows up as a collision on the next uphold. That fails in the loud direction, and the log line tells the operator to look up the derived id on-chain.
- **Found, not fixed (a pre-existing Epic 3 issue):** the settler submits every step's automatic rating under the same job_id. When one agent serves two steps, the ledger refuses the second step's rating as a replay. It's the same class of collision this PR fixes for dispute ratings, and it's out of scope here.

History notes

- One commit holds two tests because a split aborted partway.
- One commit pushed a failing test, which the next commit fixes.

Both are left in place rather than rewriting pushed history.

Verification

- ruff check, ruff format --check and mypy: clean.
- pytest --cov: **1976 passed, 2 skipped, 92.17%** (floor 82). The 4.03 base was 1880, so **+96 tests**.
- check_contract_drift.py: PASS.
- Every race test was checked by temporarily breaking the thing it guards and confirming it fails: removing the generation check fails 4, and a refund call planted on the rating path fails 6.

Orizon Agents — Week 3 Tranche Submission · Proof of Deliverables · Stellar testnet

7 / 30

What a dispute receipt needs in order to be truthful

The record behind the buyer's receipt: the amount actually credited, when the dispute last changed, the platform's reason when a dispute is rejected, and — the one that decides whether a receipt can lie — an explicit **confirmation flag on the rating**.

A transaction hash alone proves nothing, because a hash is also recorded when a submission times out. So the interface is not allowed to treat a hash as success: the rating reads as done only once the ledger has vouched for it, and the hardening pass later made that confirmation monotonic so a later timeout cannot un-confirm it or be paired with it.

Source

<https://github.com/B10cksmiths/Orizon-Agents-BE-Stellar/pull/65>

Captured

2026-09-26 19:48 PHT · screenshots/04-be-pr-65-receipt-record.png

The screenshot shows a GitHub pull request for the repository 'B10cksmiths/Orizon-Agents-BE-Stellar'. The pull request is titled '4.06 (backend) — What a dispute receipt needs to be truthful #65' and is merged. It was created by 'ALGOREX-PH' 4 days ago. The pull request includes a conversation, 92 commits, 1 check, and 14 files changed. The main content of the pull request is a document titled 'What a dispute receipt needs in order to be truthful'.

4.06 (backend) — What a dispute receipt needs to be truthful #65

ALGOREX-PH merged 92 commits into main from feat/4.06-dispute-receipt-record 4 days ago

Conversation 0 · Commits 92 · Checks 1 · Files changed 14

ALGOREX-PH commented 5 days ago

Backend half of 4.06 (BLO-34) dispute status and the refund receipt. The frontend half is a separate PR in the FE repo.

Merge and deploy this one first. The frontend receipt reads four new fields and treats their absence as "not known", so an older backend never makes it crash. But until this is deployed, every credit shows as a promise and every rating as pending.

Four facts the record couldn't give a buyer

Field	Why the receipt needs it
credited_usdc	What the refund actually transferred . credited_usdc is the promise frozen at opening, while the transfer is the minimum of three values (ADR 0008 D4), so they can differ. A receipt printing the promise beside a Stellar Expert link showing a different amount would contradict its own evidence.
rating_confirmed	Whether the dispute rating landed . 4.04 records a hash on SUCCESS and on TIMEOUT, so the hash alone can't prove the agent was rated. It's True on SUCCESS or on a replay confirming an earlier attempt, and False while only in flight.
updated_at	When the dispute last changed. resolved_at is stamped once, at the first decision, so a refund reconciled hours later had no time of its own. It's stamped on every row, by our own clock.
rejection_reason	Why a dispute was rejected, shown to the buyer.

The three new columns go in with idempotent ALTER TABLE ... ADD COLUMN IF NOT EXISTS, since CREATE TABLE IF NOT EXISTS does nothing to the table production already has. They're all nullable, so rows written before this read as "not known".

A rejection must now explain itself

The adjudicator's note was audit-only and optional. It is now **required and buyer-facing**, per the card: "a rejection with no explanation is worse than no dispute system."

- **At the route:** a missing, null or empty note is a 422, with the bound taken from the service's own constant.
- **At the service:** a note that's empty once cleaned (whitespace or control characters only) is refused with rejection_reason_required before anything is written. This also covers the in-process path the operator script uses.
- **In the API:** It's exposed only as rejection_reason, and only when the status is rejected. It's never logged.

Nothing has ever been rejected in production, because rejecting requires the refund path, which has never been switched on. So redefining the note changes no existing record.

Privacy, stated plainly: rejection_reason is as readable as the buyer's own reason. While TASK_AUTH_REQUIRED is off, as in production, that means anyone with the task id. The frontend showing it only to the payer is a display choice, not access control. The code and docs say so.

A bug found and fixed during integration

A released refund claim kept its old transaction hash. Suppose a transfer timed out (its hash recorded), was reconciled as never settled, and was released. The release, and the claim on the next attempt, both copied that hash forward. So for the whole retry, the buyer's receipt linked a **transaction that FAILED** as the refund in flight. A release only happens when nothing landed, and a claim starts a payout with no transaction yet, so neither now now carries a refund hash. The dead hash stays in the append-only history. Two tests cover it:

- one walks the exact sequence in both stores, and fails if the fix is reverted;
- one parses the SQL and asserts refund_tx is the only newly-cleared column, so a column that stops being carried by accident still fails it.

The operator script's reconciliation hints now also record credited_usdc and rating_confirmed. Without that, a credit reconciled by hand would show on the receipt as a promise forever.

Verification

- ruff check, ruff format --check, mypy, check_contract_drift.py: all exit 0.
- pytest --cov: **2049 passed, 2 skipped, 92.29%**.

History note

Several commits in the record lane don't pass on their own: the store's fake pool and a few service tests caught up a commit or two later. The branch head passes, and nothing was rewritten.

ALGOREX-PH added 30 commits 5 days ago

- added the actually credited amount to the dispute record 3f52a0e
- added the last transition time to the dispute record f62b77a
- added the rating confirmation to the dispute record e8c8ba2
- added the dispute receipt record fields b58a465
- added the actually credited amount to the dispute response 4435fa2
- added the last transition time to the dispute response ac6965a
- added the rating confirmation to the dispute response 3f55f08
- added the receipt columns to the dispute events schema 2e6ee3d
- exposed the rejection reason on a rejected dispute 8a571d4
- read the receipt columns back onto the dispute record 453a88f
- required the rejection note as buyer-facing text 63f36d5
- stamped the opening row with its own opened time 7843a0b
- refused a rejection without its note at the edge 578c785

The dispute action on the buyer's trace view

The buyer's side of D3: a settlement view on the trace page showing what each step cost and what is creditable, a live countdown to the window closing, and a Dispute button on each step still open to one. Only the paying wallet is offered the action. A visitor who is not the payer sees no dispute affordance at all and is never sent the complaint text — the server withholds it and the client blanks it independently, so no component can leak it by picking the wrong object.

Source	https://github.com/BIOcksmiths/Orizon-Agents-FE-Stellar/pull/68
Captured	2026-09-26 19:48 PHT · screenshots/05-fe-pr-68-dispute-action.png

Platform · Solutions · Resources · Open Source · Enterprise · Pricing

Search /Z Sign in Sign up

BIOcksmiths / Orizon-Agents-FE-Stellar Public

Notifications Fork Star

< Code

Issues 12

Pull requests 7

Actions

Projects

Security and quality

Insights

4.05 — Dispute action on the trace / receipt view #68

Merged

ALGOREX-PH merged 119 commits into main from feat/4.05-dispute-action 5 days ago

Conversation 1

Commits 119

Checks 4

Files changed 20

+5,433 -4

ALGOREX-PH commented 5 days ago

Collaborator

Frontend half of 4.05 (BLO-33). A buyer can now dispute a step from the page where the evidence is: the trace view of the workflow they paid for.

Needs BIOcksmiths/Orizon-Agents-FE-Stellar#64 deployed to show the receipt. It's safe to merge either way. Against an older backend (no settlement key, or a 404), the receipt stays hidden and the trace renders exactly as it does today. It never crashes.

What the buyer sees

A receipt panel above the Trace and Artifact tabs on every settled workflow. It shows what was charged, the payer, the charge and seal transactions, and the dispute window, open with a live countdown or closed with the time it closed. Below that, each step shows its state:

Step	Shown
Settled, window open, you paid	Dispute button, plus what an upheld dispute would credit
Already disputed	Its status ("Under review", "Refunded", ...). Only the payer sees their own reason
Nothing was charged	Price struck through, and "Nothing was charged for this step, so there is nothing to dispute"
Window closed, or you didn't pay	No action of any kind, and no disabled button

Not connected yet: the receipt shows, plus "if you paid for this workflow, connect that wallet to dispute a step." A connected wallet that *isn't* the payer sees no prompt and no action.

The dispute form shows, in order: the step and what it produced, the charge and the credit, and the credit terms, taken from the backend's policy (the fraction, funded by the platform and never clawed back from the agent, decided by the platform with no on-chain arbitration). Then the reason, which is mandatory with a 500-character counter. Then a note that the wallet will sign a message, which costs nothing and sends no transaction.

How it's built

Piece	What it owns
lib/disputes.ts	Every rule in the card, as a pure disputeView(), plus the API calls, response checks and the challenge → sign → submit flow. 100% covered
lib/use-dispute-panel.ts	Fetching, correcting for the server's clock, the connected wallet, and the countdown timer. It's the only clock, and it re-renders only the section that calls it
components/disputes/receipt-panel.tsx	Presentational. Draws the view model and decides nothing
components/ui/dialog.tsx	A new reusable dialog on the native <dialog>; focus is returned on close, the scroll lock works with this app's root scroller, and it can't be closed mid-signature
components/disputes/dispute-dialog.tsx	The form and its state machine: idle → signing → submitting → done or error, with a message per refusal code
components/disputes/dispute-section.tsx	Composition. The dialog is lazy-loaded, which saves 34 KB for every visitor who has nothing to dispute

The trace page itself changed by **9 lines**. An e2e test proves it **doesn't re-render per countdown tick**. With 30 minutes left and the section re-rendering every second, the trace page and its rows rendered 0 times. Adding a one-second interval to the page makes that test fail, so it does catch the problem.

Decisions worth reviewing

- Disputing uses the paying wallet's signature, not the task token the card mentions.** 4.02 settled this (ADR 0007 D2): the token proves you have the link, not that you paid. The backend accepts both signature formats real wallets produce, and this is the same flow the bind page already runs with Freighter.
- The challenge is checked before the wallet signs it.** The message must name the job and step the buyer asked about. A wallet shouldn't sign a message the client didn't expect.
- A backend 403 Access denied is never shown as "you cancelled".** The wallet-refusal detector matches /denied/, so the form only checks for a cancel on errors the wallet itself throws. A test pins that exact case.
- The receipt's step list comes from the durable settlement record, not the trace.** The trace is in memory and lost on restart, while a buyer has 24 hours to dispute.
- A non-payer's view has the complaint text removed from the data itself,** not just hidden by the panel.

Fixed along the way

The trace log's exec label measured **4.49:1** contrast, just under AA's 4.5:1. It was already wrong before this story: the existing axe sweep just never waited for those lines to render. It now uses the readable violet token.

Verification

Check	Result
tsc, lint, prettier --check	clean
vitest --coverage	1269 passed (was 981), 96.73 / 93.2 / 95.38 / 98.65
playwright (includes axe)	127 passed, 0 failed, 0 flaky. The dispute and ally specs also passed 105/105 under --repeat-each=3
axe	0 violations, including new scans of the receipt and the open dialog
bundle	trace page 411 KB / 480 KB

Every acceptance criterion has its own Playwright test. There are also tests for the full sign-and-submit path, a duplicate dispute from another tab, the window closing while the page is open, a 360px layout with no horizontal scroll, and both old-backend shapes.

All e2e evidence runs against a mocked backend. It is test evidence, not the SOW §6.1 recording, which

The dispute status and refund receipt the buyer reads

The receipt: the dispute's status, the refund transfer and the negative rating, each with its transaction hash and a Stellar Expert link a reviewer can check without trusting us.

The rule throughout is that **nothing reads as done until the chain says so**. A credit recorded whose transfer has not confirmed wears “Refund in progress”, never “Refunded”, and the amount is marked as a promise rather than a payment until the record carries what was actually paid.

Source
Captured

<https://github.com/B10cksmiths/Orizon-Agents-FE-Stellar/pull/69>
2026-09-26 19:49 PHT · screenshots/06-fe-pr-69-dispute-receipt.png

Platform · Solutions · Resources · Open Source · Enterprise · Pricing

Search /Z Sign in Sign up

B10cksmiths / Orizon-Agents-FE-Stellar Public

Notifications Fork Star

< Code

Issues 12

Pull requests 7

Actions

Projects

Security and quality

Insights

4.06 — Dispute status and refund receipt display #69

Merged ALGOREX-PH merged 71 commits into main from feat/4.06-dispute-receipt 4 days ago

Conversation 1 Commits 71 Checks 4 Files changed 12 +3,454 -94

ALGOREX-PH commented 5 days ago

Frontend half of 4.06 (BLO-34). A buyer can now follow a dispute from "Under review" to the credit landing on-chain, without leaving the trace page.

Needs B10cksmiths/Orizon-Agents-BE-Stellar#65 deployed to show its best state. Safe to merge either way: against an older backend every credit reads as a promise and every rating as pending — never as a success the chain hasn't confirmed.

What the buyer sees

A disputed step's row becomes a full receipt: the status, when it was raised and last changed, one plain sentence about what happens next, the amount with **who funded it on the same line**, and each on-chain artifact — the refund transfer and the dispute rating against the agent — with its own confirmation state and a Stellar Expert link.

And it updates itself. While a dispute is unresolved the receipt polls: every 30 s while it's under review, every 5 s while a credit is being sent, and not at all once it's final or while the tab is hidden. So during the evidence recording, the receipt flips from "Under review" to "Refunded" with both links **without a reload**, while the operator runs the uphold script.

The rule this story turns on

Nothing may read as done until the chain says so. It's enforced in the type, not by copy: an artifact is `confirmed`, `pending` or `none`, and only `confirmed` may look like success.

- `credited` is only ever written on an on-chain SUCCESS, so the refund is trustworthy.
- A rating hash proves nothing** — 4.04 records one on a timeout too — so a rating is confirmed only when the ledger vouched for it, and an older backend that can't say reads as pending.
- The badge follows the confirmation, not the status.** A dispute recorded `credited` whose transfer isn't confirmed now reads "Refund in progress", not a green "Refunded" above a sentence saying it's unconfirmed. That's a pure rule in `lib/`, tested there and at the component, and the component test fails if the badge goes back to the raw status.

How it's built

Piece	Owns
<code>lib/disputes.ts</code>	<code>disputeReceipt()</code> and <code>receiptBadgeStatus()</code> — every rule, pure. 100% covered
<code>lib/use-dispute-panel.ts</code>	The polling: cadence by status, paused when hidden, refetching at once on return, never flashing a loading state. 100% covered
<code>components/disputes/dispute-receipt.tsx</code>	Presentational. Draws the view model and decides nothing

Hashes are shown **in full**, in monospace and wrapping. In a recording a link can't be inspected, so the painted characters are the only thing a reviewer can match against Stellar Expert — a truncated hash proves 16 characters. `txStatus` is deliberately not reused: its Build → Sign → Broadcast trail describes a transaction the user signs, and the settler signs these. Its `pending/confirmed` visual language is matched instead.

A status change is announced once through a polite live region, and a poll that changes nothing announces nothing. A `MutationObserver` test counts exactly one write across five re-renders.

Verification

Check	Result
<code>tsc, lint, prettier -check</code>	clean
<code>vitest --coverage</code>	1420 passed (was 1269), 96.84 / 93.52 / 95.54 / 98.69
<code>playwright</code> (includes axe)	142 passed, 0 failed, 0 flaky, 0 skipped ; the receipt and ally specs also pass 93/93 under <code>repeat-each=3</code>
axe	0 violations, including new scans of a credited and a rejected receipt
bundle	trace page 420 KB / 480 KB

Every acceptance criterion has a Playwright test, plus the live flip, polling stopping once final, the tab-hidden pause, an old backend with none of the new fields, a non-payer seeing neither reason, and 360 px with two full hashes and no horizontal scroll.

The credited-amount test uses a `credited_usdc` that **differs** from `creditable_usdc`, so it can only pass on the amount actually transferred.

All e2e evidence runs against a mocked backend with fixture hashes that exist on no ledger. It is test evidence, not the SOW \$6.1 recording.

Known limits

- An older backend that confirms a transfer but reports no figure** shows "Up to 0.027 USDC credited to your wallet". The money did move; only the exact amount is unknown.
- The loading skeleton doesn't reserve the taller receipt's height**, so there's a small shift on first paint for a disputed step. The existing layout-shift test still passes.

ALGOREX-PH added 30 commits 5 days ago

added the optional dispute receipt fields to the dispute type ✓ 3ac356f

added the dispute receipt view types ✓ 5c274fe

added the dispute receipt to the disputed step state ✗ 36c47c8

derived the dispute receipt from the record, the viewer and the policy ✗ a27638d

added the refund and rating tx fixtures ✗ d6a72c8

attached the receipt to every disputed step ✗ bcc8e8c

added the rejection reason fixture ✓ 88e42dc

tested the receipt on each disputed step and its privacy for non-payers ✗ 6de65dc

Orizon Agents — Week 3 Tranche Submission · Proof of Deliverables · Stellar testnet

10 / 30

Backend hardening: the money path, disclosure and settlement ratings

Before calling Epic 4 done we attacked our own implementation, on the rule that nothing counted as a finding until it had been reproduced against the running service, and every fix carries a test that was written first and watched failing.

What it fixed: a **reproducible double payment** (two transfers for one dispute, 0.10 USDC out against a record saying 0.05); a stale decision that could strand a live payout's claim and leave a dispute unpayable; a missing signing key that parked a dispute mid-payment; the buyer's own words and the platform's rejection reason being readable by a caller who had proved nothing; a challenge-table flood that could deny every buyer their dispute window; a boot failure that printed configuration secrets into the deploy log; and a rating collision that lost half the ratings when one agent served two steps of a plan. Gate on merge: 2,177 tests, 92.53% coverage, type checking clean across 106 modules.

Source
Captured

<https://github.com/BI0cksmiths/Orizon-Agents-BE-Stellar/pull/75>
2026-09-26 19:49 PHT · screenshots/07-be-pr-75-hardening.png

The screenshot shows a GitHub pull request interface for the repository 'Blocksomsmiths/Orizon-Agents-BE-Stellar'. The pull request is titled '4.07 — Epic 4 hardening: dispute money path, disclosure and settlement ratings #75' and is in the 'Merged' state. It shows 113 commits, 3 checks, and 43 files changed. The pull request was merged by 'ALGOREX-PH' on 'yesterday'.

The main content of the pull request is a detailed description of the changes, organized into sections: Money, Disclosure, Reputation, Operator tooling and docs, Gate, and Not in this PR. The 'Money' section lists several changes related to dispute handling, including a new 'settlement' state, a 'refund' state, and a 'dispute' state. The 'Disclosure' section lists changes related to dispute visibility, including a new 'dispute' state, a 'dispute' state, and a 'dispute' state. The 'Reputation' section lists changes related to dispute ratings, including a new 'dispute' state, a 'dispute' state, and a 'dispute' state. The 'Operator tooling and docs' section lists changes related to dispute tooling, including a new 'dispute' state, a 'dispute' state, and a 'dispute' state. The 'Gate' section lists changes related to dispute gatekeeping, including a new 'dispute' state, a 'dispute' state, and a 'dispute' state. The 'Not in this PR' section lists changes related to dispute handling, including a new 'dispute' state, a 'dispute' state, and a 'dispute' state.

The pull request also includes a list of commits and a list of files changed. The commits are listed in a table with columns for the commit message, the commit hash, and the commit date. The files changed are listed in a table with columns for the file name, the file size, and the file type.

Frontend hardening: the receipt stops stating what it cannot show

An upheld dispute used to read “the credit is being sent to your wallet”, over a row captioned “what you received”, marked “Being sent” — with nothing submitted and no transaction in existence. A buyer would go looking on Stellar Expert for a transfer nobody had attempted. All three lines now state only what the record holds.

Also fixed: a single 404 on any re-read silently destroyed a live receipt with no error and no recovery; “Nothing on this workflow was charged” was terminal and never re-checked; the countdown overstated the window by the whole network round trip, offering a Dispute button the server would refuse; and the moment a refund confirms was announced to nobody using a screen reader. Gate on merge: 1,503 unit tests, 149 end-to-end tests, every route inside the 480 KB budget.

Source
Captured

<https://github.com/BI0cksmiths/Orizon-Agents-FE-Stellar/pull/76>
2026-09-26 19:49 PHT · screenshots/08-fe-pr-76-hardening.png

The screenshot shows a GitHub pull request interface. At the top, the repository is 'BI0cksmiths/Orizon-Agents-FE-Stellar'. The pull request title is '4.07 (frontend) — Epic 4 hardening: the receipt stops stating what it cannot show #76'. It was merged by 'ALGOREX-PH' yesterday. The pull request description details a hardening pass over the Epic 4 frontend, mentioning 26 defects fixed. It lists several key changes: the receipt now states 'The credit has not been sent yet' instead of 'the credit is being sent to your wallet'; the on-chain caption now says 'what you received' instead of 'what you are owed'; the mark 'Being sent' now reads 'No transaction on record'; the refund sentence now says 'The refund was submitted' instead of 'The refund was submitted' when 'refund_tx' is null; the panel 'Nothing on this workflow was charged' is now terminal; a single 404 on any re-read destroyed a live receipt; the clock overstated the window; the dispute with an unconfirmed refund never re-read; a dispute from a different job was pinned onto this settlement's step; and one unrecognised row blanked the whole receipt. The 'Accessibility' section lists improvements like keyboard focus, fallback pointing, and screen reader announcements. The 'Consistency' section notes changes to percentage rounding, agent name rendering, and error messages. The 'Gate' section shows test results: 1503 tests across 50 files, 99.9% coverage, and a 480 KB first-load budget. The 'Depends on' section lists the backend half of the pull request. The commit history shows 30 commits by 'ALGOREX-PH' yesterday. The file diff shows changes to 'formatCreditShare' and 'agentLabel' definitions, and various test files.

Backend pull requests merged this week

Every backend pull request merged in the sprint week, filtered by merge date in GitHub's own search. Stories 4.02 through 4.06 and the 4.07 hardening pass all landed on main.

One record in that list is an honest mistake worth naming: PR #61 was opened against an unmerged feature branch rather than main, so its commits did not reach main until PR #62 re-landed them. Both are in the public record, and 05-pull-requests.md in this bundle explains why the two share a diff.

Source	https://github.com/BIOcksmiths/Orizon-Agents-BE-Stellar/pulls?q=is%3Apr+is%3Amerged+merged%3A2026-09-21..2026-09-26
Captured	2026-09-26 19:49 PHT · screenshots/09-be-pull-requests-week3.png

Platform

Solutions

Resources

Open Source

Enterprise

Pricing

Search

Sign in

Sign up

BIOcksmiths / Orizon-Agents-BE-Stellar

Public

Notifications

Fork 0

Star 1

<> Code

Issues 17

Pull requests 8

Actions

Projects

Security and quality

Insights

All pull requests

New pull request

is:pr is:merged merged:2026-09-21..2026-09-26

Open 0 Closed 8

AuthorLabelProjectsMilestonesReviewsAssigneeNewest

4.07 — Epic 4 hardening: dispute money path, disclosure and settlement ratings

#75 · ALGOREX-PH opened yesterday · Closed yesterday Collaborator · ✓ 1/1

4.06 (backend) — What a dispute receipt needs to be truthful

#65 · ALGOREX-PH opened 5 days ago · Closed 4 days ago Collaborator · ✓ 1/1

4.05 (backend) — Settlement view for the dispute receipt

#64 · ALGOREX-PH opened 5 days ago · Closed 5 days ago Collaborator · ✓ 1/1

4.04 — Negative on-chain rating for an upheld dispute

#63 · ALGOREX-PH opened 5 days ago · Closed 5 days ago Collaborator · ✓ 1/1

4.03 — Settler-executed partial-credit refund (land on main)

#62 · ALGOREX-PH opened 5 days ago · Closed 5 days ago Collaborator · ✓ 1/1

4.03 — Settler-executed partial-credit refund

#61 · ALGOREX-PH opened 5 days ago · Closed 5 days ago Collaborator · ✓ 1/1

1

4.02 — Dispute window state and the dispute endpoint

#60 · ALGOREX-PH opened 5 days ago · Closed 5 days ago Collaborator · ✓ 1/1

Untrack evidence/ops docs, keep dev docs only

#42 · ALGOREX-PH opened 2 weeks ago · Closed 4 days ago Collaborator · ✓ 1/1

© 2026 GitHub, Inc.

Terms

Privacy

Security

Status

Community

Docs

Contact

Manage cookies

Do not share my personal information

Frontend pull requests merged this week

Every frontend pull request merged in the sprint week. Stories 4.05 and 4.06 shipped the buyer-facing dispute surface; PR #76 is the hardening pass over it.

The two Week-2 evidence documents (PRs #65 and #66) also appear here because they merged on the Monday; their work was authored the previous week and is counted there.

Source	https://github.com/BIOcksmiths/Orizon-Agents-FE-Stellar/pulls?q=is%3Apr+is%3Amerged+merged%3A2026-09-21..2026-09-26
Captured	2026-09-26 19:49 PHT · screenshots/10-fe-pull-requests-week3.png

Platform Solutions Resources Open Source Enterprise Pricing

Search / Sign in Sign up

BIOcksmiths / Orizon-Agents-FE-Stellar Public

Notifications Fork 0 Star 1

<> Code Issues 12 Pull requests 7 Actions Projects Security and quality Insights

All pull requests New pull request

is:pr is:merged merged:2026-09-21..2026-09-26

Open 0 Closed 6

Author Label Projects Milestones Reviews Assignee Newest

Week 3 tranche submission — Deliverable D3 evidence bundle

#83 · ALGOREX-PH opened 28 minutes ago · Closed 25 minutes ago Collaborator · 4/5

1

4.07 (frontend) — Epic 4 hardening: the receipt stops stating what it cannot show

#76 · ALGOREX-PH opened yesterday · Closed yesterday Collaborator · 5/5

1

4.06 — Dispute status and refund receipt display

#69 · ALGOREX-PH opened 5 days ago · Closed 4 days ago Collaborator · 5/5

1

4.05 — Dispute action on the trace / receipt view

#68 · ALGOREX-PH opened 5 days ago · Closed 5 days ago Collaborator · 5/5

1

Week 2 technical documentation and demo evidence PDF

#66 · ALGOREX-PH opened last week · Closed 5 days ago Collaborator · 5/5

1

Week 2 proof of deliverables: add the summary pages

#65 · ALGOREX-PH opened last week · Closed 5 days ago Collaborator · 5/5

1

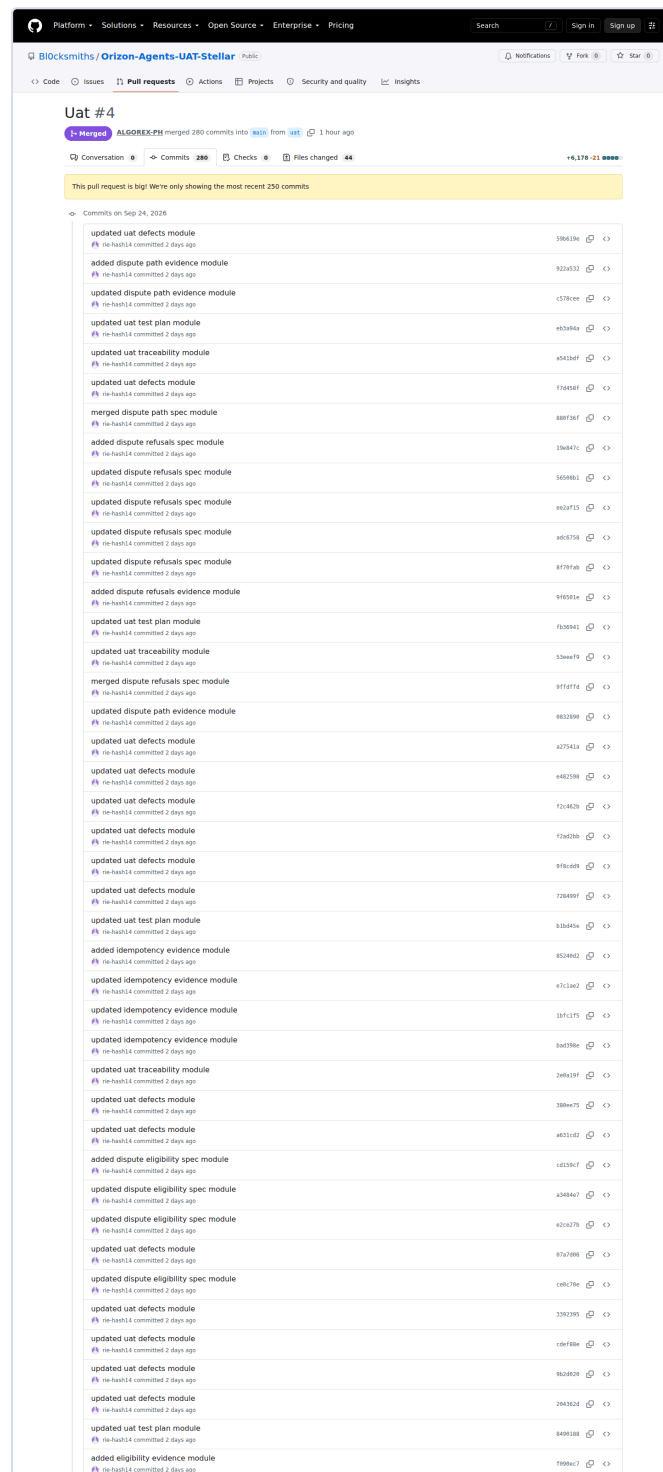
© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Independent QA: 268 authored commits in the public UAT repository

QA is a separate pair of eyes and a separate public repository. The commits tab of UAT PR #4 shows **rie-hash14** as the author of every commit in it — 268 authored commits this week plus 12 merges, each touching exactly one file.

The work behind them: the seven 6.03 sub-stories (the dispute happy path, attack idempotency on the money path, who may dispute and when, durability and the unconfirmed-refund path, the reputation consequence, the dispute UI in every state, and the adjudication door), the drills that produced the two testnet transactions later in this document, and the defect register.

Source <https://github.com/BI0cksmiths/Orizon-Agents-UAT-Stellar/pull/4/commits>
 Captured 2026-09-26 19:49 PHT · screenshots/11-uat-pr-4-rie-commits.png



The live service is on testnet, against the deployed contracts

The deployed backend reports the network it is on and the contract ids it is wired to. Both are unchanged from Week 2: Epic 4 needed no contract change, because the dispute rating is written through the ReputationLedger already deployed.

Note the asset: the configured SAC is the **native XLM SAC on testnet**, while the interface labels amounts in USDC. Choosing the refund asset deliberately is one of the outstanding items before the D3 evidence run, and it is listed as such in this bundle rather than left for a reviewer to notice.

Source	https://orizon-agents-be-stellar.onrender.com/api/stellar/network
Captured	2026-09-26 19:49 PHT · screenshots/12-be-stellar-network.png

GET https://orizon-agents-be-stellar.onrender.com/api/stellar/network — fetched 2026-09-26T11:49:53.447Z (response body, indented for legibility)

```
{
  "network": "testnet",
  "rpc_url": "https://soroban-testnet.stellar.org",
  "network_passphrase": "Test SDF Network ; September 2015",
  "admin": "GA7AISTAJEZA27I666DSJC4MUJYBEWUYNZWPUR2ONA7IZQV06R50QV",
  "dispatch_signer": "GB5MKHDFLJZ60FPAHM7R4HGBUPFV5PZYL3W27VTIUZZ25JMQSDZBKCMR",
  "asset": "native",
  "asset_sac": "CDLZFC3SYJDZT7K67VZ75HPJVIEUVNIXF47ZG2FB2RMQQVU2HHGCYSC",
  "contracts": {
    "agent_registry": "CAPHXWU53UZUZJGV7IAE57NNMH3YYB5MTW06YA53KKMXSFVLOITBJ3GQ",
    "reputation_ledger": "CDCS0BEVZUPQZV5GV4D6KYHZCLNGW2KXY74RUHSZ3EZUXF34DPW422ZT",
    "payment_escrow": "CBJPTMAPMG0DGZC22IMEQSRUX3WGUXNMKDTNN2KMJ3NFGYZ50J5525PI",
    "attestation_registry": "CBYUZZK0ET43UXTBZUJIBBJW50DGD2J2AZVXXCR3QONGOCAHOXQQHEGK"
  }
}
```

The platform's signer is the ledger's authorised scorer

The one live precondition for D3 that already holds: `ratings.writer` reads "scorer", meaning the account the deployment signs with is the account the ReputationLedger authorises to write ratings. Without that, an upheld dispute would refund the buyer and fail to record the consequence.

The same probe is how we verify it after every deploy, and it is checked by the six-hourly production smoke test.

Source	https://orizon-agents-be-stellar.onrender.com/readiness
Captured	2026-09-26 19:49 PHT · screenshots/13-be-readiness.png

GET https://orizon-agents-be-stellar.onrender.com/readiness — fetched 2026-09-26T11:49:54.261Z (response body, indented for legibility)

```
{
  "status": "ready",
  "llm": "ok",
  "stellar": "configured",
  "signer": "configured",
  "pdax": "configured",
  "cold_start": {
    "routable": true,
    "lower_bound_bps": 5677,
    "floor_bps": 5500,
    "margin_bps": 177
  },
  "ratings": {
    "writer": "scorer",
    "signer": "GDB4N25UYM3YNTTAWX7LSGI2P70R62QZQXRNQWAGF5TFVENDKCTTCDHP",
    "scorer": "GDB4N25UYM3YNTTAWX7LSGI2P70R62QZQXRNQWAGF5TFVENDKCTTCDHP"
  }
}
```

All six dispute routes are served by the deployed API

The deployment serves the whole Epic 4 surface: the challenge mint, opening a dispute, reading one dispute, reading a task's disputes, and the adjudication pair that upholds or rejects. That is six routes under the disputes tag; QA's own notes count five, because they leave out the per-task read that the trace page uses. Same surface, two ways of counting it.

The same description also shows that the deployment is running a build from **before** the hardening merge — the operator security scheme that BE #75 adds is absent from it. That is stated plainly in this bundle: the hardened code is on main, not yet on the deployed service.

Source <https://orizon-agents-be-stellar.onrender.com/openapi.json>

Captured 2026-09-26 19:50 PHT · screenshots/14-be-dispute-routes-openapi.png

Orizon Agents API 0.1.0 OAS 3.1

/openapi.json

The orchestration layer for autonomous digital labor: goal decomposition across a priced agent network, Soroban-settled payments and reputation on Stellar, and PDAX fiat on/off-ramping. Errors use a unified envelope — the legacy "detail" key plus an "error" object with a stable code, message, and request id.

[Orizon Agents - Website](#)

Servers

<https://orizon-agents-be-stellar.onrender.com> - production

Interactive docs at <https://orizon-agents-be-stellar.onrender.com/docs> — the other eleven tag sections and the Schemas block are hidden so the six dispute routes fit one frame.

disputes Dispute a settled step inside its 24-hour window, proved by the payer's wallet signature; and the operator-keyed adjudication that upholds one into a credit or rejects it. ^

POST /api/disputes/challenge Mint a dispute challenge to sign

POST /api/disputes Open a dispute on a settled step

GET /api/disputes/{dispute_id} Read one dispute

GET /api/tasks/{task_id}/disputes A task's dispute window and the disputes raised on it

POST /api/disputes/{dispute_id}/uphold Uphold a dispute and credit the buyer

POST /api/disputes/{dispute_id}/reject Reject a dispute, with a reason the buyer is shown

The refund path is switched off on the deployment

Asking the deployed service to uphold a dispute answers 503 `dispute_refunds_disabled`. The refund path ships behind a master switch that is **off by default**, deliberately: a money path enabled by the mere presence of a signing key would be enabled in every test run and on every developer's laptop, which is how an anonymous payout route reaches production without anyone choosing it.

The gate covers the whole adjudication pair, not just the paying half: POST `.../reject` answers the same 503, so on this deployment a dispute cannot even be refused. PR #62's own deploy notes state the shipped default — the path needs both the switch and an operator key set in the host dashboard before it will move money.

This is the current, honest state of the deployment rather than a demonstration. Turning the switch on is one step of the outstanding work for D3 — and not the first one, because a deeper defect means no workflow settles there to be disputed in the first place.

Source <https://orizon-agents-be-stellar.onrender.com/openapi.json>

Captured 2026-09-26 19:50 PHT · screenshots/15-be-refunds-disabled.png

Live state of the dispute money path

fetch() from the browser against <https://orizon-agents-be-stellar.onrender.com> — captured 2026-09-26T11:50:00.964Z

REQUEST

```
POST https://orizon-agents-be-stellar.onrender.com/api/disputes/00000000-0000-0000-0000-000000000000/uphold
Content-Type: application/json

{"reason": "evidence capture – refusal expected"}
```

RESPONSE

```
503

{
  "detail": "dispute_refunds_disabled",
  "error": {
    "code": "dispute_refunds_disabled",
    "message": "dispute refunds disabled",
    "request_id": "51e49bedbd804722"
  }
}
```

The deployment refuses to uphold a dispute: refunds are switched off, so no buyer credit is attempted. The gate answers before the dispute id is looked up, so any id returns this same body. The id above is a placeholder — no dispute can exist on the deployment, because the settlement record a dispute hangs off is never written (BE issue #67). This is the current live state, not a fault staged for the screenshot.

A real refund transfer on testnet — from QA's drill, not the deployment

QA ran the **real backend code** through one uphold and it signed a real transfer on testnet, confirmed on the ledger and re-read on Horizon. It proves the refund path executes end to end and produces a verifiable transaction.

It is not the Deliverable-3 evidence, and this document does not present it as such. The asset is a test asset, not USDC; the ledger is QA's own drill ReputationLedger, not the platform's; and the source account is a developer machine's, not the deployment's signer GDB4N25U...CDHP — which anyone can check on this very page. D3 stays open until one run on the deployed service produces this transaction.

Source <https://stellar.expert/explorer/testnet/tx/a5baac432b582787df0a632b3bc12916c51e12575a8f77bf267fd45b728701b8>
Captured 2026-09-26 19:50 PHT · screenshots/16-drill-refund-tx-stellar-expert.png

The screenshot displays the Stellar Expert testnet explorer interface. At the top, the navigation bar includes 'stellar expert', 'NETWORK STATS', 'ASSETS', 'MARKETS', 'SERVICES', and 'BLOG'. The 'Network' dropdown is set to 'testnet'. The main heading shows the transaction ID: 'Transaction a5baac432b582787df0a632b3bc12916c51e12575a8f77bf267fd45b728701b8'. Below this, the 'Summary' section provides details: Status is 'Successful', Ledger is '4865810', Source Account is 'GA45IT...RJEGZ2', Sequence Number is '20881383668842554', Transaction size is '772 bytes', and Preconditions are 'Valid before 2026-09-25 16:10:58'. Processing details show it was processed on '2026-09-25 16:10:37 UTC' with a 'Max Fee' of '0.0024308 XLM' and a 'Fee Charged' of '0.0014515 XLM'. The transaction details section shows 'GA45...EGZ2 invoked contract CCW6...UENL transfer(GA45...EGZ2, GBXF...5XTD, 1000000.1128)'. The 'Signatures' section shows the signature for GA45IT...RJEGZ2. The footer contains version information '2026 © StellarExpert v1.0.2', links for 'Open API docs', 'Report a bug', 'Dark theme', 'Terms of use', 'Privacy policy', and 'Status', and a donation address.

The matching negative rating on testnet — same drill, same caveat

The second half of the consequence, one ledger after the refund: the kind="dispute" rating written against the agent whose step was disputed. Refund and rating are the two artifacts D3 promises, and both were produced by one uphold.

Same caveat, stated again because it matters: this was signed against a drill ledger from a developer machine. It demonstrates that the code works; it does not demonstrate the deployment doing it.

Source	https://stellar.expert/explorer/testnet/tx/7138e4e36e47f4f4404b2212aad5584d2f8fb941b387da76acae4c3b4cc07184
Captured	2026-09-26 19:50 PHT · screenshots/17-drill-rating-tx-stellar-expert.png

stellar expert

NETWORK STATSASSETS MARKETS SERVICES BLOG

Network testnet

Transaction 7138e4e36e47f4f4404b2212aad5584d2f8fb941b387da76acae4c3b4cc07184

Summary

Status: Successful

Ledger: 4865811

Source Account: GA45IT...RJE6Z2

Sequence Number: 20881383668842555

Transaction size: 1,080 bytes

Preconditions

Valid before 2026-09-25 16:11:06

Processed: 2026-09-25 16:10:42 UTC

Max Fee: 0.0081693 XLM

Fee Charged: 0.0064578 XLM (0.001444 non-refundable + 0.0050038 refundable + 0.00001 min op fee)

GA45...EGZ2 invoked contract CAFZ...5CRP submit(GA45...EGZ2, "agt_0915" sym, zYk2Gpe1LiZiEZuTB4bvLg== bytes, 10 u32, 1000000 1128, GBXF_5XTD, "dispute" sym)

Signatures

GA45IT...RJE6Z2: oG0baM1IP62LjyGk32ktBl2AjdAWwyRhRzNmyUjsSjVMrThT5Kz9TYc6z6S/xesYEF5QR4YPBCrmmr6qdAQ==

2026 © StellarExpert v1.0.2

[Open API docs](#) [Report a bug](#) [Dark theme](#)

[Terms of use](#) [Privacy policy](#) [Status](#)

[Twitter](#) [Email](#)

Donations: GDQT5AG9VSH3ZHUZPATVADQSDN9K36XJGJ2VQHM57NMC5TQDEP

Why no dispute can be raised on the deployment today

The honest centre of this week's report. A dispute window is stamped on a **settlement** — and no run on the deployment records one, so there is nothing to dispute. QA found and filed it: a finished, paid run leaves the settlement record unwritten, the dispute endpoint answers that the job is unknown, and nothing in the product says so.

The whole of Epic 4 is therefore unreachable by a buyer on the deployed service, even though it is built, merged and tested. We would rather a reviewer read that here than discover it themselves.

Source

<https://github.com/Blocksmiths/Orizon-Agents-BE-Stellar/issues/67>

Captured

2026-09-26 19:50 PHT · screenshots/18-be-issue-67-no-settlement.png

Platform · Solutions · Resources · Open Source · Enterprise · Pricing

Search / Sign in Sign up

Blocksmiths/Orizon-Agents-BE-Stellar public

Notifications Fork Star

< Code Issues Pull requests Actions Projects Security and quality Insights

No finished run is disputable: the settlement record is never written, so the dispute path is unreachable #67

Open New issue

rie-hash14 opened 2d ago

Severity: Critical

Reproduce

Run any workflow to completion on the deployed service, then read its disputes. From the 2026-09-24 QA re-run, `tsk_e534ce3029393aee` (status `complete`, spent 8.834, both steps delivered):

```
curl -s https://orizon.nyz/api/tasks/tsk_e534ce3029393aee/disputes
```

Expected

A settlement to dispute against and a closing time: `dispute_window_seconds` (default 86 400) is stamped onto the settlement record at settle time, and the trace is meant to announce `dispute_window_open`: any delivered step can be disputed until: ...

Actual

```
("task_id":"tsk_e534ce3029393aee","window_closes_at":null,"now":"1798228036.5","settlement":null,"disputes":[])
```

No trace in seven runs carried a dispute-window line. `...record:settlement` returns early when the charge produced no `job_id` (`app/renderer/execute.py:39:389:382.1`), and the charge never lands because `PaymentEscrow.charge` cannot move the payer's funds (filed separately against the contract repo). Seven authorizations produced 0 `charge` events.

Impact

`POST /api/disputes`, the challenge and adjudication routes, the refund executor, the negative-rating path and the whole `/api/trace` dispute UI cannot be reached by any buyer on testnet. Nothing in the product says so: the task still reads `complete` with a non-zero `spent`.

Suggested fix

Primarily the escrow fix. Until then, consider writing a settlement row (or telling the buyer why there is none) when a charge fails, so the state is visible rather than silent.

Note

Epic 4's dispute and refund work has no QA acceptance criteria yet; UAT added EX-09 to cover reachability only.

Found in QA story 6.05 (external agent execution path), UAT defect D-050, re-verified on the 2026-09-24 re-run. Evidence: <https://github.com/Blocksmiths/Orizon-Agents-UAT-Stellar/blob/uat/docs/uat/evidence/6.05-external-dispatch.md> · log: <https://github.com/Blocksmiths/Orizon-Agents-UAT-Stellar/blob/uat/docs/uat/defects.md>

rie-hash14 2d ago Author

6.03a — still blocked (re-checked 2026-09-24 13:28 UTC)

I didn't start the session, both blockers from this morning are unchanged, and running it now would waste the one continuous recording the story needs.

- Dispute routes are deployed. All five are in the live OpenAPI (create, challenge, read, uphold, reject), so the 2026-09-22 "zero dispute routes" note is out of date.
- Refunds are switched off on the deploy (D-051 → [Dispute refunds are disabled on the deployed service](#), so no dispute can be upheld (blocks story 6.03a) #65, blocker). `POST /api/disputes/123/uphold` answers "set dispute refused: disabled". `DISPUTE_REFUND_DISABLE` and a non-empty `API_KEY` need setting in Render. Until then, `uphold_dispute.py` would be refused even with a dispute open.
- Nothing can be disputed (D-050 → [No finished run is disputable: the settlement record is never written, so the dispute path is unreachable #67](#), which follows from D-019 → [PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs Orizon-Agents-Smart-Contract-Stellar#3, Critical](#)). The escrow charge never lands, so no settlement is recorded. That means no receipt panel and no stop to dispute.
- Tests: the dispute-path and dispute-refunds specs pass against the live stack (52 passed). DP-01 is among them because it's marked as an expected failure. It turns green by itself once a charge lands.

No wallet was signed, no recording was started and no transaction was submitted. The re-check is recorded in [docs/uat/evidence/6.03a-dispute-path.ad](#) §7 (Blocksmiths/Orizon-Agents-UAT-Stellar · uat · 0832890).

To unblock, in order:

- Fix and redeploy the escrow (charge authorization) [PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs Orizon-Agents-Smart-Contract-Stellar#3](#). DP-01 going green confirms it.
- In Render, set `DISPUTE_REFUND_DISABLE` and `API_KEY`, and confirm `DATABASE_URL` (`DATABASE_URL` can't be checked from outside).
- Book one sitting with Dan, who holds the settler key. Rie runs the buyer wallet and the recording; Dan runs the dry run, then the live uphold.

rie-hash14 1d ago Author

Story 6.03d (durability and the unconfirmed-refund path) — status 2026-09-25: the live half is blocked by this issue.

Re-checked 07:43 UTC. [#6-8](#): still fails as expected, so no paid run has a settlement record, and there is still no dispute on the deploy to restart around. Render restarted the service by itself during the check (`/health` uptime 9.5 s), so restarts are routine and the live test only needs a settlement to exist.

Criterion	Result
DU-01 a dispute survives a restart	Holds locally — not run live
DU-02 a settlement stays disputable after a restart	Holds locally — not run live
DU-03 an unconfirmed refund reads as pending	Holds locally — not run live
DU-04 a stuck payout is findable and not retried	Fails: #77 (D-064)
DU-05 the durable store is in use	Holds on the deploy. <code>DATABASE_URL</code> is set. Related: #76 (D-063)

What fixing this unblocks: DU-01 and DU-02 live. [#6-8](#): turning green is the signal that this can start. It still depends on [Blocksmiths/Orizon-Agents-Smart-Contract-Stellar#3](#).

Correction to the 2026-09-24 comment above: `DATABASE_URL` can now be confirmed from outside, and it is set. Evidence: [Blocksmiths/Orizon-Agents-UAT-Stellar](#) [docs/uat/evidence/6.03d-durability.ad](#) (UAT `uat` c605594).

rie-hash14 9h ago Author

Story 6.03 status (2026-09-26): still blocked on this issue

Re-checked against <https://orizon-agents-be-stellar.srender.com>:

- [#6-8](#): still fails as expected. No finished run has a settlement record, so no step can be disputed. The live 6.03 specs pass 30/30 on `(chronik-disklog)`, their expected failures included.
- The deploy still runs a build older than `88afda`: `/api/agents.json` has no security scheme.
- The contracts repo has had no commit since 2026-09-24. D-039 [PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs Orizon-Agents-Smart-Contract-Stellar#3](#) still stands behind this issue.

What this blocks: the 6.03 card's first criterion, SD-01: a USDC refund and a kind-of-dispute rating from one upheld dispute, both resolving on Stellar Expert. It also blocks the Deliverable 3 evidence: the dispute tx, the refund tx and the

The root cause, at the contract level

One layer down: `PaymentEscrow.charge` attempts the transfer from an account that never signs, so the buyer's funds do not move and the run cannot settle. This is a contract defect, not configuration, and it is the gating item for D3 — until it is fixed, no amount of deployment or environment work produces a disputable run.

It is public, reproducible and filed against the contracts repository. Sequencing the remaining D3 work behind it is the first thing `03-deliverable-D3-dispute-refund.md` in this bundle does.

Source
Captured

<https://github.com/Blocksmiths/Orizon-Agents-Smart-Contract-Stellar/issues/3>
2026-09-26 19:50 PHT · screenshots/19-contracts-issue-3-escrow-charge.png

Platform · Solutions · Resources · Open Source · Enterprise · Pricing

Search / Sign In Sign Up

Blocksmiths / Orizon-Agents-Smart-Contract-Stellar Public

Notifications Fork Star

Code Issues Pull requests Actions Projects Security and quality Insights

PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs #3

New Issue

Open

Severity: Critical

What happens

authorize records an Authorization, takes no custody and grants no allowance. charge is settler-gated (caller.require_auth()) and ends in `usdc:transfer(auth.payer, agent.owner, amount)`. The SAC's transfer requires `frank.require_auth()` — the payer's authorization — but the charge transaction is signed by the settler alone, so the host rejects it with `Error(Auth, InvalidAction)`.

Evidence from two QA runs against testnet

Contract `CB3PTNPMG06J2722ME0SRUCMUNWNTN2N2N3W6V25025528FE`:

- 2026-09-17: 9 authorizations, 0 charged events.
- 2026-09-24 (after the backend redeploy; contract unchanged): 7 more authorizations, 0 charged events. The payer's balance moved only by transaction fees.

Every run still finalizes as `complete` with a `settler` figure and `charge.tx` null, and the buyer's trace carries a single `error`: `on-chain settlement failed line`.

Impact

No operator has ever been paid through the escrow. The escrow's existing successful charges are the settler paying itself, which cannot stand in for a buyer paying an operator. It also makes the whole Epic 4 dispute and refund path unreachable, since a settlement record is only written when a charge lands (filed separately against the backend).

Suggested fix

Take custody of the payer's asset at `authorize` time, or have `charge` use `transfer.from` against an allowance the payer granted, or have the payer co-sign the charge. The deployed ABI has no `approve`, `transfer.from` or `deposit` endpoint, so closing this needs a new contract deployment.

Found in QA story 6.05 (external agent execution path), UAT defect D-039, re-verified on the 2026-09-24 re-run. Evidence: <https://github.com/Blocksmiths/Orizon-Agents-UAT-Stellar/blob/uat/docs/uat/evidence/6.05-external-dispatch.md> · log: <https://github.com/Blocksmiths/Orizon-Agents-UAT-Stellar/blob/uat/docs/uat/defects.md>

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Participants

Open in GitHub Copilot app

6.03a — still blocked (re-checked 2026-09-24 13:28 UTC)

I didn't start the session. Both blockers from this morning are unchanged, and running it now would waste the one continuous recording the story needs.

- Dispute routes are deployed. All five are in the live OpenAPI (create, challenge, read, uphold, reject), so the 2026-09-22 "zero dispute router" note is out of date.
- Refunds are switched off on the deploy (D-051 — ☐ Dispute refunds are disabled on the deployed service, so no dispute can be upheld (blocks story 6.03a) Orizon-Agents-BE-Stellar#68, Blocker). POST `/api/disputes/145/upheld` answers 303 dispute-refunds disabled, `DISPUTE_REFUNDS_ENABLED=true` and a non-empty `API_KEY` need setting in Render. Until then, `uphold_dispute.py` would be refused even with a dispute open.
- Nothing can be disputed (D-050 — ☐ No finished run is disputable: the settlement record is never written, so the dispute path is unreachable Orizon-Agents-BE-Stellar#67, which follows from D-039 — ☐ PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs #3, Critical). The escrow charge never lands, so no settlement is recorded. That means no receipt panel and no step to dispute.
- Tests: the dispute-path and dispute-refunds specs pass against the live stack (52 passed). DP-01 is among them because it's marked as an expected failure. It turns green by itself once a charge lands.

No wallet was signed, no recording was started and no transaction was submitted. The re-check is recorded in `docs/uat/evidence/6.03a-dispute-path.md` §7 (Blocksmiths/Orizon-Agents-UAT-Stellar · uat · 0832890).

To unblock, in order:

- Fix and redeploy the escrow charge authorization (☐ PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs #3). DP-01 going green confirms it.
- In Render, set `DISPUTE_REFUNDS_ENABLED=true` and `API_KEY`, and confirm `DATABASE_URL` · `DATABASE_URL` can't be checked from outside.
- Book one sitting with Dan, who holds the settler key: Rie runs the buyer wallet and the recording; Dan runs the dry run, then the live uphold.

Story 6.03d (durability and the unconfirmed-refund path) — status 2026-09-25: the live half is blocked, and this issue is the root of it.

Re-checked 07:43 UTC: ☐-R1 still fails as expected. The escrow charge never lands, so no paid run has a settlement record (Blocksmiths/Orizon-Agents-BE-Stellar#67), and there is no dispute on the deploy to restart around. Render restarts the service by itself routinely (1.heartbeat, uptime was 9.5 s during the check), so the live test needs only a charge that lands.

Criterion	Result
DU-01 a dispute survives a restart	Holds locally — not run live
DU-02 a settlement stays disputable after a restart	Holds locally — not run live
DU-03 an unconfirmed refund reads as pending	Holds locally — not run live
DU-04 a stuck payout is findable and not retried	Fails: Blocksmiths/Orizon-Agents-BE-Stellar#77 (D-064)
DU-05 the durable store is in use	Holds on the deploy: <code>DATABASE_URL</code> is set. Related: Blocksmiths/Orizon-Agents-BE-Stellar#76 (D-063)

What fixing this unblocks: a charge that lands writes the settlement, which clears Blocksmiths/Orizon-Agents-BE-Stellar#67. That makes DU-01 and DU-02 runnable live: open a dispute on a settled run, let Render restart the service, reload the trace page, then dispute a second step. ☐-R1, turning green is the signal.

Evidence: Blocksmiths/Orizon-Agents-UAT-Stellar · docs/uat/evidence/6.03d-durability-1.txt · uat · c605594.

Story 6.03 status (2026-09-26): this issue is still the root blocker

Re-checked today:

- The contracts repo has had no commit since 2026-09-24. With no escrow charge landing, the backend writes no settlement record (☐ No finished run is disputable: the settlement record is never written, so the dispute path is unreachable Orizon-Agents-BE-Stellar#67). So no finished run can be disputed, and ☐-R1 in the UAT suite still fails as expected.
- The backend deploy still runs a build older than `18efed6`, and refunds are still off (☐ Dispute refunds are disabled on the deployed service, so no dispute can be upheld (blocks story 6.03a) Orizon-Agents-BE-Stellar#68). That issue now lists the steps before refunds can go on.

What this blocks: the 6.03 card's first criterion, SD-01: a USDC refund and a `kind="dispute"` rating from one upheld dispute, both resolving on Stellar Expert. It also blocks the Deliverable 3 evidence: the dispute tx, the refund tx and the dispute tx recording. A local drill proved the backend path on testnet (refund `spbaas8j888`, rating `7388863388`). It used a test asset on the drill's own ledger, so it can't stand in for Deliverable 3.

Once a charge lands on the deploy, ☐-R1 turns green by itself. That is the signal that the 6.03 live session can be

The defect register: 27 defects logged this week

QA logged **27 defects** this week (D-050 to D-076): one Blocker, two Critical, seven Major, seventeen Minor. Twenty-five are filed as public GitHub issues against the repository that owns the code. Two money-path defects are held privately — their mechanisms were handed to the key holder and the maintainers directly rather than published while the deployment cannot be patched, which is the correct handling.

Eleven of the 27 were found against **our own hardening build**, which is what independent QA is for. One of them, D-067, is a regression our disclosure fix introduced: a payer who reopens their trace in a new tab loses their own dispute reason. QA's verdict on the 6.03 story card is **no-go**, and this bundle reports it as no-go.

Source
Captured

<https://github.com/BIOcksmiths/Orizon-Agents-UAT-Stellar/blob/main/docs/uat/defects.md>
2026-09-26 19:50 PHT · screenshots/20-uat-defect-register.png

Platform • Solutions • Resources • Open Source • Enterprise • Pricing

Search / Sign in Sign up

Blocksmiths / Orizon-Agents-UAT-Stellar Public

Code Issues Pull requests Actions Projects Security and quality Insights

Files

main

Go to file

ci

docs

evidence

uat

checklists

evidence

defects.md

dispatch-6.02.md

runbook.md

signoff-report.md

test-plan.md

traceability.md

wallet-browser-matrix.md

live-site-findings.md

tests

tools

envuat.example

gitignore

README.md

package-lock.json

package.json

playwright.config.ts

tsconfig.json

Orizon-Agents-UAT-Stellar / docs / uat / defects.md

rie-hash14 updated uat defects module ca9db09 · 5 hours ago History

Preview Code Blame 3111 lines (2348 loc) · 148 KB Raw

UAT defects

Severity definitions are in `test-plan.md`.

D-001 — Target deployment is mainnet while the programme is specified testnet-only

- Severity: Blocker
- Status: Open
- Affects: EV-01, EV-02, EV-03, EV-04

Steps to reproduce

```
curl -s https://orizons.xyz/api/stellar/network
```

Expected — "network": "testnet", matching the QA task's "Week 1 (M1) · testnet only" scope and its `--api-base https://orizons.xyz` verification command.

Actual

```
"network": "mainnet",
"rpc_url": "https://mainnet.sorobanrpc.com",
"network_passphrase": "Public Global Stellar Network ; September 2015"
```

The four contract ids returned are the mainnet deployment.

Impact — The registration evidence journey requires signing a real registration. On this target that is a live mainnet transaction spending real XLM against non-upgradable contracts. No test may execute it, so EV-01 through EV-04 cannot be exercised end to end and are recorded Blocked rather than failed.

Resolution path — Point `UAT_BASE_URL` at a deployment whose `/api/stellar/network` reports testnet, then re-run the EV suite. The suite reads the expected network from `UAT_EXPECTED_NETWORK` rather than hardcoding it, so no test change is needed when the target flips.

D-002 — Cannot install dependencies or execute any test on the authoring machine

- Severity: Blocker (process, not product)
- Status: Partly resolved — see "Update" below
- Affects: the verification step of every criterion

Steps to reproduce — Run `npm ci` in this repo on the authoring machine.

Expected — Dependencies install; `npm test` runs.

Actual — Three attempts, three failures: two OOM kills (at 689 and 230 packages) and one `ENOTEMPTY` file lock on `node_modules/nest/compat`. The machine reports 3.83 GB total RAM with 0.35-0.78 GB free. A subsequent recursive delete of `node_modules` also failed.

Impact — No commit in this programme has been runtime-verified. Every commit was checked statically only: brace balance, import resolution, and a line-for-line diff against the authored source. That static pass did catch a real defect (two spec files silently truncated during chunked commits), but it cannot catch a wrong selector or a failing assertion.

Resolution path — Either free memory on the authoring machine (pausing real-time antivirus is the highest-value single step; the failed delete and the install slowness both point at it), or add `workflow` scope to the GitHub token so CI becomes the verifier. See D-003.

Update — the frontend half now runs. `npm ci` succeeded on the fourth attempt in a fresh clone of the frontend repository, with no partial `node_modules` to contend with: 1234 packages in 18 minutes under `--max-old-space-size=1024`. All seven prepared frontend fix branches were applied there and the toolchain executed:

check	result
<code>npm run typecheck</code>	pass
<code>npm run lint (--max-warnings=0)</code>	no warnings or errors
<code>npm run test:coverage</code>	500 tests, 27 files, all passing
coverage	96.71 / 93.88 / 96.40 / 98.74 vs thresholds 86 / 78 / 88 / 89

The first execution **failed**, on two type errors introduced by the fixes themselves — a changed return type that no longer satisfied `useBallot`'s callback signature, and a renamed state setter left behind in one branch of `skillsInput.tsx`. Both had survived every static pass. The second is the sharper lesson: the grep meant to catch the rename was case-sensitive and so could never have matched `setRejected`. The verification had a hole shaped exactly like the bug it was supposed to find. This is the concrete cost this defect was logged to warn about, and it is now measured rather than predicted.

A second gap the coverage report exposed. Two of the fixed modules had no test at all — `lib/settlement-asset.ts` at 0% and `lib/wallet-picker-ally.ts` (the D-017 keyboard-trap fix) at 64% statements / 33% branches — despite the handoff notes claiming every branch carried its own regression test. Both are now covered: 11 tests and 13 tests respectively, taking the first module to 100% across statements, branches and functions and the second to 100 / 91.66 / 100 / 100. Passing thresholds had been hiding two entirely untested modules.

What is still blocked. This repository's own Playwright suite — the 190 test blocks that are the substance of the programme — has still never been installed or run. Nothing below the frontend unit layer has been executed: no `pytest` against the backend, no `cargo test` against the contracts. The sign-off recommendation is unchanged.

D-003 — CI cannot run: the GitHub token lacks workflow scope

- Severity: Critical
- Status: Open
- Affects: the "CI green on uat" exit criterion

Steps to reproduce — Commit a file under `.github/workflows/` and push.

Actual

```
1 [remote rejected] refusing to allow a Personal Access Token to create or
update workflow '.github/workflows/e2e.yml' without 'workflow' scope
```

Impact — The suite has no automated runner. The workflow is complete but parked at `ci/e2e/workflow.yml`, where GitHub ignores it. Combined with D-002, nothing in this programme has executed anywhere.

Resolution path — Add `workflow` scope to the token, then `git mv ci/e2e/workflow.yml .github/workflows/e2e.yml`. The workflow already covers three browser projects, a backend warm-up step, and report artifacts.

D-004 — Skills validation messages are unreachable

The dispute action, with the window open

Captured locally against stubbed API responses, not from the deployed service. The dispute surface cannot be reached on the deployment (see the two defects above), so these six images show the interface that shipped this week, rendered from the fixtures the end-to-end suite uses. Every amount and hash in them is a fixture that exists on no ledger.

What it shows: the settlement view on the trace page — what each step cost, what is creditable under the policy, a live countdown to the window closing, and a Dispute button on each step still open to one.

Source <https://github.com/BI0cksmiths/Orizon-Agents-FE-Stellar/blob/main/components/disputes/receipt-panel.tsx>
Captured 2026-09-26 19:42 PHT · screenshots/local-01-dispute-action-open-window.png

Receipt ✓ SETTLED

Settled 1h ago · 3 steps

TOTAL CHARGED

0.063 USDC

PAYER

GBRPYHIL2CI3FNQ4BXLFMNDLFJUNPU2HY3ZMFSHONUCE0ASW7QC70X2H

[VIEW PAYER ON STELLAR.EXPERT >](#)

SETTLED

Sep 26, 2026, 6:38 PM GMT+8

CHARGE

a41c7e0d93b25f6817ce4a0b9d3f72e15c86a0d4b7e2f91c3a58d06e4b1f7c29

[VIEW CHARGE ON STELLAR.EXPERT >](#)

SEAL

0e9d4c71b3a85f2e6c1d07b94a3e8f52d6c10a7e9b4f38d2c5a16e0b7d93f4a8

[VIEW SEAL ON STELLAR.EXPERT >](#)

DISPUTE WINDOW OPEN

22h 59m Left

closes Sep 27, 2026, 6:38 PM GMT+8

TERMS

An upheld dispute credits 50% of that step's charge back to you, paid by the platform and never clawed back from the agent. The platform decides each dispute; there is no on-chain arbitration.

STEPS

01

seo.brief

outline with 12 target keywords

0.009 USDC

DISPUTE

credits 0.0045 USDC if upheld

02

code.gen

calculator app, 3 files

0.054 USDC

DISPUTE

credits 0.027 USDC if upheld

03

vision.ocr

Nothing was charged for this step, so there is nothing to dispute.

0.012 USDC

NOT CHARGED

Orizon Agents — Week 3 Tranche Submission · Proof of Deliverables · Stellar testnet

25 / 30

Raising a dispute on one step

The form names the step being disputed, states the share of that step's charge an upheld dispute would credit, and takes the buyer's reason. Submitting asks the wallet to sign a one-time challenge; nothing was signed for this capture.

Fixture capture, as above — the step, the agent and the amounts come from the test suite's fixtures.

Source <https://github.com/B10cksmiths/Orizon-Agents-FE-Stellar/blob/main/components/disputes/dispute-dialog.tsx>
Captured 2026-09-26 19:39 PHT · screenshots/local-02-dispute-dialog.png

DISPUTE

×

Dispute step 2

Ask the platform to credit part of what this step cost.

▶ THE STEP

Step 2 · **code.gen**

calculator app, 3 files

▶ WHAT IS AT STAKE

CHARGED FOR THIS STEP

0.054 USDC

CREDITED IF UPHELD

0.027 USDC

▶ THE TERMS

- An upheld dispute credits 50% of this step's charge back to the wallet that paid.
- The platform pays the credit. Nothing is clawed back from the agent.
- The platform reviews the dispute and decides. There is no on-chain arbitration.

▶ YOUR REASON

Required. Say what went wrong with this step — the platform reads this when it decides.

the calculator app does not compute anything

CANCEL

SIGN AND SUBMIT

An open dispute, waiting on the platform

The receipt while a dispute is open: the status the buyer is in, and one sentence saying what happens next. No refund row and no rating row, because neither exists yet — an absent artifact on an open dispute is not news, and the receipt does not pretend otherwise.

Fixture capture, as above.

Source	https://github.com/B10cksmiths/Orizon-Agents-FE-Stellar/blob/main/components/disputes/dispute-receipt.tsx
Captured	2026-09-26 19:41 PHT · screenshots/local-03-receipt-open-dispute.png

02

code.gen

0.054 USDC

calculator app, 3 files

DISPUTE RECEIPT

● UNDER REVIEW

Raised Sep 26, 2026, 6:59 PM GMT+8 · 40m ago Updated Sep 26, 2026, 7:09 PM GMT+8 · 30m ago

The platform is reviewing this dispute; if it is upheld, the step's credit is paid to your wallet and code.gen's reputation records the dispute.

CREDIT

 · Up to 0.027 USDC would be credited to your wallet if upheld — funded by the platform, not clawed back from the agent.

YOUR REASON

the calculator app does not compute anything

A credited dispute: refund and rating, both confirmed

What a completed dispute looks like to the buyer: the amount credited, the refund transfer with its transaction and explorer link, and the negative rating written against the agent with its own. The two links are the evidence a reviewer checks; the receipt's job is to hand them over rather than to be believed.

Fixture capture — **the hashes shown here exist on no ledger**. The two real testnet transactions in this document are shots 16 and 17, and their caveats are stated there.

Source <https://github.com/BI0cksmiths/Orizon-Agents-FE-Stellar/blob/main/components/disputes/dispute-receipt.tsx>
Captured 2026-09-26 19:41 PHT · screenshots/local-04-receipt-credited.png

02

code.gen

0.054 USDC

calculator app, 3 files

DISPUTE RECEIPT

✓ REFUNDED

Raised Sep 26, 2026, 6:59 PM GMT+8 · 40m ago Updated Sep 26, 2026, 7:09 PM GMT+8 · 30m ago

Done: you received 0.027 USDC, and it cost code.gen a dispute rating on its reputation.

CREDIT · 0.027 USDC credited to your wallet — funded by the platform, not clawed back from the agent.

ON-CHAIN RECORD

Refund transfer

WHAT YOU RECEIVED

✓ CONFIRMED ON STELLAR

63e658669328e993706eedab30602d3da9e1538cbb008c9a6515333750984327

VIEW REFUND ON STELLAR.EXPERT ▶

Dispute rating against code.gen

WHAT IT COST THE AGENT

✓ CONFIRMED ON STELLAR

e46d4a71165f437fd4cfaa402be58dcd63163ef1cfbac4772494a27b40136b30

VIEW RATING ON STELLAR.EXPERT ▶

YOUR REASON

the calculator app does not compute anything

The state this epic exists to get right

A credit the platform has recorded whose transfer the chain has **not** confirmed. The badge reads “Refund in progress”, never “Refunded”; the row says there is no transaction on record rather than linking one; and the amount is stated as what is owed rather than what was received.

This is the single most important behaviour in Epic 4, and the one the hardening pass was needed to secure: before it, the same state read “the credit is being sent to your wallet” under a success tick. A payment interface that overstates what has happened is worse than one that says nothing, because the buyer goes looking for a transaction that does not exist. Fixture capture, as above.

Source	https://github.com/BIOcksmiths/Orizon-Agents-FE-Stellar/blob/main/lib/disputes.ts
Captured	2026-09-26 19:41 PHT · screenshots/local-05-receipt-crediting-unconfirmed.png

02

code.gen

0.054 USDC

calculator app, 3 files

DISPUTE RECEIPT

● REFUND IN PROGRESS

Raised Sep 26, 2026, 6:59 PM GMT+8 · 40m ago Updated Sep 26, 2026, 7:09 PM GMT+8 · 30m ago

The platform recorded this credit as paid, but the refund transfer is not confirmed on Stellar yet; the platform reconciles it by hand — you will not be paid twice, and will not be skipped.

CREDIT · Up to 0.027 USDC to be credited to your wallet — funded by the platform, not clawed back from the agent.

ON-CHAIN RECORD

Refund transfer

WHAT IS OWED TO YOU

● NO TRANSACTION ON RECORD

Dispute rating against code.gen

WHAT IT COST THE AGENT

✓ CONFIRMED ON STELLAR

e46d4a71165f437fd4cfaa402be58dcd63163ef1cfbac4772494a27b40136b30

VIEW RATING ON STELLAR.EXPERT ▶

YOUR REASON

the calculator app does not compute anything

A rejected dispute, with the platform's reason

A buyer whose dispute is refused is told why, in the adjudicator's own words, and no credit line is drawn. The reason is shown to the payer only: a visitor who is not the payer is never sent it.

Fixture capture, as above.

Source	https://github.com/BI0cksmiths/Orizon-Agents-FE-Stellar/blob/main/components/disputes/dispute-receipt.tsx
Captured	2026-09-26 19:43 PHT · screenshots/local-06-receipt-rejected.png

